

A Novel Incremental Class Learning Technique for Multi-class Classification

Meng Joo Er¹(✉), Vijaya Krishna Yalavarthi², Ning Wang¹,
and Rajasekar Venkatesan²

¹ Marine Engineering College, Dalian Maritime University, Dalian, China
emjer@ntu.edu.sg, n.wang.dmu.cn@gmail.com

² School of Electrical and Electronic Engineering,
Nanyang Technological University, Singapore, Singapore
{vijayakr001, raja0046}@e.ntu.edu.sg

Abstract. In this paper, a novel technique for multi-class classification, which is independent of the number of class constraints and can learn the new classes it encounters, is developed. The developed technique enables remodelling of the network to adapt to the dynamic nature of non-stationary input samples. It not only can learn the new classes, but also the new patterns created in the input. The proposed algorithm is evaluated using various benchmark datasets and a comparative study of classification performance shows that the proposed algorithm is superior.

Keywords: ELM · OS-ELM · Incremental class learning · Sequential learning · Multi-class classification

1 Introduction

Classification problems are one of the age-old problems in the computational intelligence community. In classification, the problem of grouping input sequences into one of more than two disjoint classes is called multi-class classification. Based on learning paradigms, multi-class classification problems are divided into two categories. The first one is batch learning where the entire input data are available for training which takes place throughout the availability of data once. Another one is sequential learning where the initial training takes place with available data and later, training happens continuously whenever new datum arrives.

In the recent years, much research on computation intelligence pertaining to sequential learning has been reported [1–3]. In multi-class classification, sequential learning algorithms can learn the new data of a fixed number of classes which it has been trained initially. However, whenever a new class is encountered, the network is to be retrained with the entire set of data, which makes it very time consuming. In applications like cognitive robotics, the number of classes varies and the trained data may not be available to retrain the network with the new class encountered. This makes the existing sequential learning algorithms unsuitable for these applications.

In this paper, a novel algorithm, which can adapt to the new classes it encounters and changes to the input pattern by automatically changing the architecture of the

network, is proposed. The proposed algorithm updates the network without losing the knowledge of the previous training. The proposed algorithm is different from the CIELM of [4] where it does not consider changes in the input patterns created during the addition of a new class. The Class-Incremental Learning was proposed using the support vector machines [5] where the problem is considered as binary classification. A new class is considered as one class and all the existing classes as another. The major disadvantage of this method is that the data will be unbalanced and accuracy is reduced.

The proposed algorithm is developed from the Online Sequential Extreme Learning Machine (OS-ELM) of [1]. The OS-ELM originates from the batch learning Extreme Learning Machine (ELM) of [6]. Several versions of OS-ELM and ELM are proposed in the literature [7–13]. The performance of the OS-ELM [1] is evaluated on several benchmark datasets and the results are compared with various sequential learning techniques. Simulation results show that the OS-ELM gives better generalization performance at very high learning speed. The performance of the proposed algorithm is evaluated on several standard datasets and it is compared with the CIELM and OS-ELM. In the proposed methodology, we systematically increase the number of output neurons to facilitate the learning of new classes and the hidden layer neurons to adapt to changes in the input patterns.

The paper is organised as follows: In Sect. 2, a brief review on the ELM and OS-ELM is discussed. This is followed by the proposed algorithm in Sect. 3. Simulation results are discussed in Sect. 4 and conclusions are drawn in the last section.

2 Background

In this section, a brief review of the ELM and OS-ELM is presented.

2.1 Extreme Learning Machine

Let us assume that the data have N samples represented by $\{(x_i, y_i)\}_{i=1}^N$ where x_i is the input vector and y_i is the output vector. Consider the hidden layer with P neurons. Then, the output can be given as follows:

$$f_P(x) = \sum_{j=1}^P \beta_j G(w_j, b_j, x) = y \quad (1)$$

As per the theory of ELM [6], input weights w and hidden neurons bias b are randomly determined and the output weights β are obtained by $\beta = H^+ Y$ where H^+ is the Moore-Penrose generalized inverse of the hidden layer output matrix and $Y = [y_1, \dots, y_N]^T$.

2.2 Online Sequential Extreme Learning Machine

In the OS-ELM, it is assumed that the data arrive sequentially. The OS-ELM has two phases of training [1].

Initialisation Phase: In this phase, the OS-ELM works like the batch learning ELM with an initial data N_0 , represented by $\{(x_{0i}, y_{0i})\}_{i=1}^{N_0}$ and H_0 is the hidden layer output matrix for the initial training section N_0 . The output weight vector β^0 is calculated using

$$\beta^0 = M_0^{-1} H_0^T Y_0 \quad (2)$$

where $M_0 = H_0^T H_0$ is stored for the sequential learning phase.

Sequential Learning Phase: After the initialisation phase, data arriving sequentially are used to train the network. In this phase, the output weight vector β^{k+1} is calculated as the function of the previous output weight vector β^k , partial hidden layer output matrix H_{k+1} and the output vector Y_{k+1} . Let k be 0 for the first block. The output weight vector β^{k+1} is calculated as follows:

$$\beta^{k+1} = \beta^k + M_{k+1}^{-1} H_{k+1}^T (Y_{k+1} - H_{k+1} \beta^k) \quad (3)$$

where $M_{k+1} = M_k + H_k^T H_k$.

3 Proposed Algorithm

The proposed algorithm works like OS-ELM until a new output class arrives. Whenever a new output class or set of classes arrive, the output layer of the network will be updated with the addition of nodes based on the number of new classes. Along with the output layer, the hidden layer is also updated with the addition of new nodes to adapt to the pattern changes in the input data.

Consider an initial dataset containing N_0 samples represented by $Q = \{(x_Q^{(i)}, y_Q^{(i)})\}_{i=1}^{N_0}$ where $x_Q^{(i)} \in X$ is the input vector and $y_Q^{(i)}$ is the output vector with m classes. The proposed algorithm works like the OS-ELM until new classes arrive. Hence, β^0 is determined using (2). Later, the data arrive block by block and a block may have one or more samples. Let us assume that there is no new class until k^{th} block. The output weight matrix β^k is determined using (3).

Later, $(k+1)^{th}$ block with a new data of N_1 samples and Δm new classes represented by $K = \{(x_K^{(i)}, y_K^{(i)})\}_{i=1}^{N_1}$ arrive. To facilitate the new classes in the output, we add Δm new nodes in the output layer. This makes a total of m^* nodes in the output layer. There will be a change to the input pattern whenever a new class arrives. To accommodate this change, we increase the number of neurons in the hidden layer. Let ΔP nodes be added to the hidden layer and the input weights and bias are generated randomly as per the theory of OS-ELM [1]. The total number of nodes in the hidden layer be P^* . Due to the addition of new hidden nodes, M_k will be modified to M_k^* as follows:

$$M_k^* = I_{P^* \times P} * M_k * I_{P^* \times P}^T \quad (4)$$

where $I_{P^* \times P} = \begin{bmatrix} I_{P \times P} \\ 0_{\Delta P \times P} \end{bmatrix}$ is a rectangular identity matrix and $0_{\Delta P \times P}$ is a zero matrix of order $\Delta P \times P$.

All the newly added hidden layer neurons are connected to all the m^* output nodes with zero weights. Due to the introduction of new hidden layer neurons and output neurons, the output weight vector is modified to β^{k^*} which is given by

$$\beta^{k^*} = I_{P^* \times P} * \beta^k * I_{m \times m^*} \quad (5)$$

where $I_{m \times m^*} = [I_{m \times m} \ 0_{m \times \Delta m}]$ is a rectangular identity matrix and $0_{m \times \Delta m}$ is a zero matrix of order $m \times \Delta m$.

Then β^{k+1} is calculated using

$$\beta^{k+1} = \beta^{k^*} + M_{k+1}^{-1} H_{k+1}^T (Y_{k+1} - H_{k+1} \beta^{k^*}) \quad (6)$$

where $M_{k+1} = M_k^* + H_{k+1}^T H_{k+1}$.

Subsequently it will continue as the OS-ELM until new classes arrive.

In the proposed algorithm, the network restructure itself allowing the changes in the data without losing the previous information.

4 Simulation Results and Discussions

The proposed algorithm works in the dynamic environment where the number of classes is often unknown. The algorithm automatically redesigns the network for adapting new classes and the variations to the input patterns without losing the previous knowledge. The proposed algorithm is tested on five benchmark datasets which are Waveform, Image Segmentation, Satellite Image, Letter Recognition and Pen-Based Recognition of Handwritten Digits given in Table 1. All the datasets are taken from the UCI machine learning repository.

We assume that the data are balanced and uniformly distributed. Each dataset is divided into 70 % for training and 30 % for testing. Without loss of generality and for the sake of simplicity, we demonstrate the addition of one new class. The training dataset is split into two subsets where the latter has one more class of the data that were not present in the former. The first subset is used for the initial training. The number of hidden neurons is found using the training and validation method where 80 % of the data are used for training and 20 % for validation. Experiments are repeated over 50 trials and the optimal number of hidden neurons which gives best validation accuracy is used for training. We consider the ratio of hidden to output neurons as constant and whenever a new class is presented, hidden neurons are added along with the output neurons to keep the ratio same. After initial training, the number of hidden layer neurons and output neurons are increased and trained for the new class. The distribution of datasets is given in Table 1.

Table 1. Specifications of datasets used

Dataset	Attributes	Train data	Test data	Range	No. of classes	Hidden nodes
Waveform	21	3500	1500	1–705	2	100
				706–3500	3	150
Image segmentation	19	1617	963	1–416	6	150
				417–1617	7	175
Satellite image	36	4504	1931	1–1033	5	150
				1034–4504	6	180
Letter recognition	16	3247	1392	1–811	5	350
				812–3247	6	420
Pen-based recognition of handwritten digits	17	3659	1583	1–953	6	240
				953–3659	7	280

The testing accuracy curve of the Waveform data is shown in Fig. 1. Initially, the network is trained with two classes. When the third class is introduced at the 706th sample, the network updated itself by increasing the number of neurons in the output and hidden layers to adapt to the new class and input variations. Testing accuracy suddenly rises when a new class has arrived and settled down after a few iterations.

Figure 2 shows the testing accuracy for the letter recognition data in which A, B, C, D and E are the base classes. Initially, the network is trained with these five base classes. It can be seen that for in 811th sample F class is introduced and learning the new class did not affect the performance of the existing classes significantly.

Consistency is an important factor every algorithm should have. The proposed algorithm is experimented several times on each of the aforementioned datasets and the testing accuracy of all the trails is shown in Fig. 3.

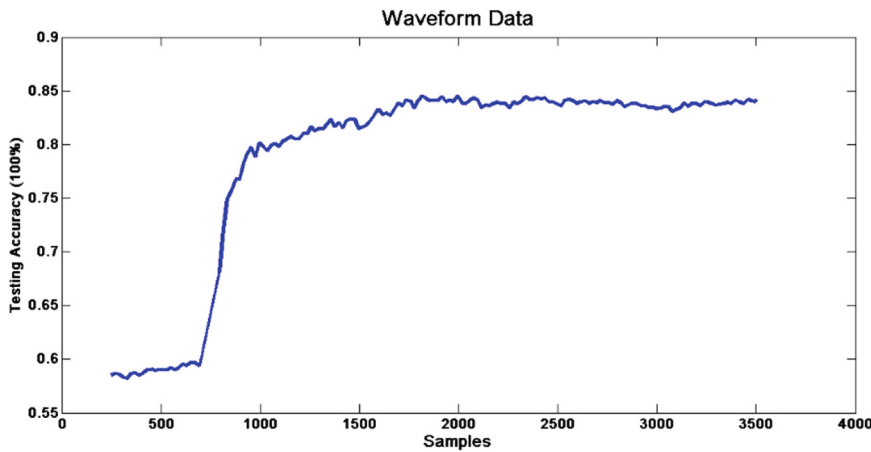


Fig. 1. Testing accuracy of waveform data

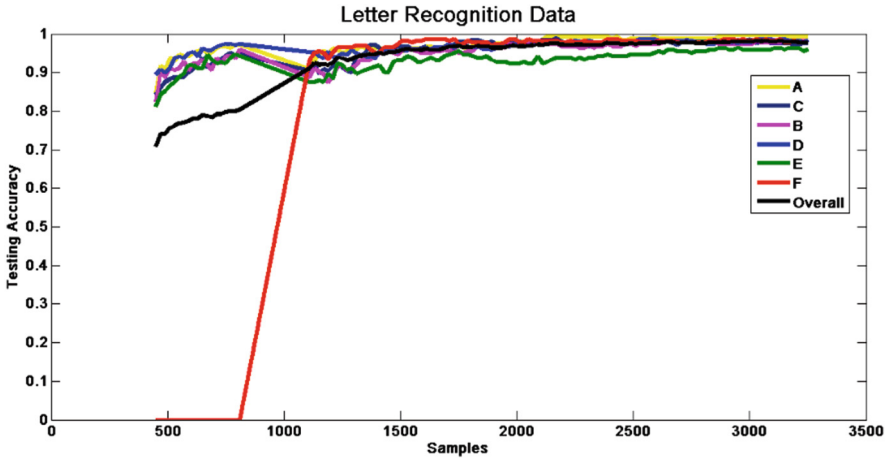


Fig. 2. Overall and individual testing accuracy for letter recognition data

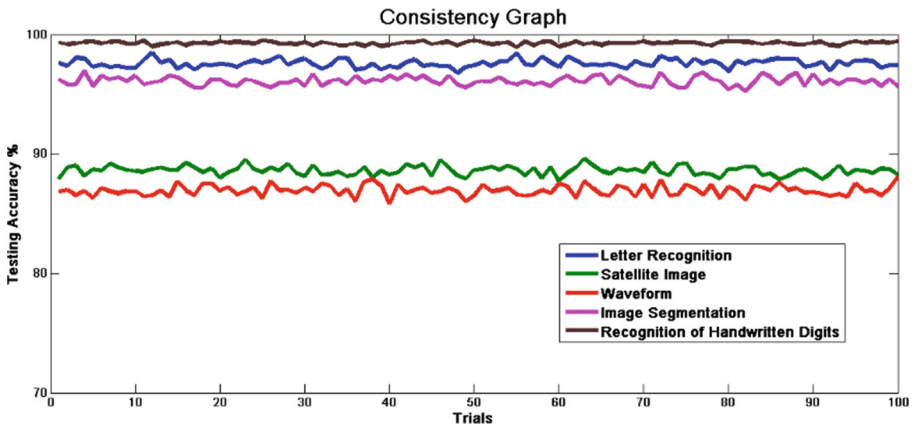


Fig. 3. Consistency of the proposed algorithm in testing accuracy

It can be observed from the graph that the proposed algorithm is consistently accurate in each of the attempts. The deviation of the testing accuracy in each case is in the order of 0–2 % of the mean value which is nominal.

The performance of the proposed algorithm is compared with the OS-ELM and CIELM algorithms and simulation results are given in Table 2. Testing accuracy along with the standard deviations for all the data sets mentioned are compared. A computer with Intel core i7 processor and 8 GB RAM was used for all the simulation works.

It can be observed from the simulation results that the testing accuracy of the proposed algorithm is better than the CIELM and at par with the OS-ELM. In summary, the proposed algorithm can learn the new class it encounters without losing the previous knowledge and it can adapt to pattern variations at the input.

Table 2. Performance comparison

Data type	Proposed algorithm testing accuracy	OS-ELM testing accuracy	CIELM testing accuracy
Waveform	85.40 ± 0.67	85.30 ± 0.68	83.90 ± 0.71
Image segmentation	95.27 ± 0.34	95.07 ± 0.73	94.94 ± 0.37
Satellite image	88.60 ± 0.37	88.93 ± 0.57	88.17 ± 0.42
Letter recognition	98.03 ± 0.31	98.50 ± 0.34	97.62 ± 0.34
Pen-based recognition of handwritten digits	99.28 ± 1.20	99.39 ± 1.10	99.18 ± 1.30

5 Conclusions

In this paper, a novel algorithm for incremental class learning is proposed. This algorithm adapts to the new output classes it encounters and variations to the input patterns by modifying the network without losing previous knowledge. The proposed algorithm is well suited to real-time applications like cognitive robotics where the number of output classes is often unknown. The performance of the proposed algorithm is compared with the OS-ELM and CIELM and the proposed algorithm shows promising results.

Acknowledgement. The authors would like to acknowledge the funding support from the National Natural Science Foundation of P. R. China (under Grants 51009017 and 51379002), Applied Basic Research Funds from Ministry of Transport of P. R. China (under Grant 2012-329-225-060), and Pro-program for Liaoning Excellent Talents in University (under Grant LJQ2013055).

References

1. Liang, N.-Y., Huang, G.-B., Saratchandran, P., Sundararajan, N.: A fast and accurate online sequential learning algorithm for feedforward networks. *IEEE Trans. Neural Netw.* **17**, 1411–1423 (2006)
2. Wang, N., Er, M.J., Han, M.: Parsimonious extreme learning machine using recursive orthogonal least squares. *IEEE Trans. Neural Netw. Learn. Syst.* **25**, 1828–1841 (2014)
3. Wang, N., Er, M.J., Han, M.: Generalized single-hidden layer feedforward networks for regression problems. *IEEE Trans. Neural Netw. Learn. Syst.* **26**, 1161–1176 (2015)
4. Zhao, Z., Chen, Z., Chen, Y., Wang, S., Wang, H.: A class incremental extreme learning machine for activity recognition. *Cogn. Comput.* **6**, 423–431 (2014)
5. Zhang, B., Su, J., Xu, X.: A class-incremental learning method for multi-class support vector machines in text classification. In: 2006 International Conference on Machine Learning and Cybernetics, pp. 2581–2585. IEEE (2006)
6. Huang, G.-B., Zhu, Q.-Y., Siew, C.-K.: Extreme learning machine: theory and applications. *Neurocomputing* **70**, 489–501 (2006)
7. Li, M.-B., Huang, G.-B., Saratchandran, P., Sundararajan, N.: Fully complex extreme learning machine. *Neurocomputing* **68**, 306–314 (2005)

8. Zhu, Q.-Y., Qin, A.K., Suganthan, P.N., Huang, G.-B.: Evolutionary extreme learning machine. *Pattern Recogn.* **38**, 1759–1763 (2005)
9. Huang, G.-B., Chen, L., Siew, C.-K.: Universal approximation using incremental constructive feedforward networks with random hidden nodes. *IEEE Trans. Neural Netw.* **17**, 879–892 (2006)
10. Li, B., Wang, J., Li, Y., Song, Y.: An improved on-line sequential learning algorithm for extreme learning machine. In: Liu, D., Fei, S., Hou, Z.-G., Zhang, H., Sun, C. (eds.) *ISNN 2007, Part I. LNCS*, vol. 4491, pp. 1087–1093. Springer, Heidelberg (2007)
11. Rong, H.-J., Huang, G.-B., Sundararajan, N., Saratchandran, P.: Online sequential fuzzy extreme learning machine for function approximation and classification problems. *IEEE Trans. Syst. Man Cybern. B Cybern.* **39**, 1067–1072 (2009)
12. Huang, G.-B., Wang, D.H., Lan, Y.: Extreme learning machines: a survey. *Int. J. Mach. Learn. Cybern.* **2**, 107–122 (2011)
13. Wang, X., Han, M.: Improved extreme learning machine for multivariate time series online sequential prediction. *Eng. Appl. Artif. Intell.* **40**, 28–36 (2015)